

Контрольні запитання

1. Що таке інтерфейс `ICrudServiceAsync<T>`? Яке його призначення? Чому доцільно використовувати generic тип `T` для CRUD-сервісу?
2. Що таке процес та потік, яка між ними різниця? Що таке багатопотокове виконання?
3. Що таке асинхроність, яка різниця між асинхронністю та багатопотоковістю?
4. Для чого використовуються ключові слова `async/await`?
5. Чим відрізняється `Task` від `ValueTask`?
6. Що таке `thread-safe` колекція? Які приклади таких колекцій у .NET ви знаєте?
7. Для чого використовуються примітиви синхронізації, такі як `lock`, `Semaphore`, `AutoResetEvent`?
8. Як забезпечити безпеку при одночасному зверненні кількох потоків до спільного ресурсу?
9. Як за допомогою LINQ можна отримати мінімальне, максимальне та середнє значення певної властивості?
10. У чому різниця між методами `Select`, `Where`, `Aggregate`, `OrderBy`?
11. Які переваги використання LINQ у порівнянні з класичними циклами?
12. Що станеться, якщо два потоки одночасно спробують зберегти колекцію у файл?
13. Як працює `Parallel.For` і у яких випадках його краще використовувати?
14. Що таке пагінація і як вона реалізована у вашій системі?

Що таке інтерфейс `ICrudServiceAsync<T>` Яке його призначення? Чому доцільно використовувати generic тип `T` для CRUD-сервісу?

Відповідь

`ICrudServiceAsync<T>` — це інтерфейс, який визначає стандартний набір асинхронних операцій життєвого циклу даних (Create, Read, Update, Delete). Його призначення — забезпечити єдиний контракт для доступу та модифікації сутностей у системі, гарантуючи неблокуюче виконання IO операцій. Доцільність використання generic типу `T` полягає в тому, що він дозволяє створити один універсальний сервіс, який може працювати з будь-яким типом сутності (наприклад, `textUser` чи `textProduct`), що значно зменшує дублювання коду та забезпечує типову безпеку.

Що таке процес та потік, яка між ними різниця? Що таке багатопотокове виконання?

Відповідь

Процес — це екземпляр програми, що виконується; він має свій ізольований простір пам'яті та системні ресурси. Потік — це базова одиниця виконання в межах процесу; потоки ділять пам'ять процесу. Головна різниця полягає в ізоляції: процеси незалежні, тоді як потоки спільно використовують ресурси, що робить їхнє спілкування швидшим, але

вимагає синхронізації. Багатопотокове виконання — це одночасне або паралельне виконання кількох потоків у межах одного процесу, що використовується для підвищення продуктивності textCPU-інтенсивних завдань.

Що таке асинхронність, яка різниця між асинхронністю та багатопотоковістю?

Відповідь

Асинхронність — це механізм, що дозволяє виконувати textIO-операції (наприклад, роботу з мережею чи файлами) неблокуюче. Потік, який ініціював операцію, звільняється, щоб виконувати іншу роботу, поки чекає на результат. Багатопотоковість — це паралелізм (одночасне виконання) за допомогою кількох потоків для textCPU-bound завдань. Різниця: Асинхронність фокусується на ефективному використанні одного потоку під час очікування textIO, тоді як багатопотоковість фокусується на паралельному виконанні обчислень з використанням багатьох потоків.

Для чого використовуються ключові слова textasyncawait?

Відповідь

Ключові слова textasyncawait використовуються для спрощення написання асинхронного коду в text.NET. Ключове слово textasync позначає метод, який може містити оператор textawait. Оператор textawait призупиняє виконання textasync-методу в точці виклику textIO операції, звільняючи поточний потік. Після завершення textIO операції виконання методу відновлюється, що дозволяє коду виглядати лінійно і уникнути "пекла зворотних викликів" (callback hell).

Чим відрізняється textTask від textValueTask?

Відповідь

textTask — це Reference Type (клас), що завжди виділяє пам'ять у textHeap для представлення результату асинхронної операції, навіть якщо операція завершується синхронно. textValueTask — це Value Type (структура), яка не виділяє пам'ять у textHeap, якщо асинхронна операція завершується синхронно (наприклад, дані вже є в кеші). textValueTask використовується для мікрооптимізації та зменшення навантаження на textGarbage Collector у

високопродуктивних бібліотеках, де частіше очікується синхронний результат.

Що таке `textthread-safe` колекція? Які приклади таких колекцій у `text.NET`?

Відповідь

`textThread-safe` колекція — це структура даних, доступ до якої з кількох потоків одночасно є безпечним і не призводить до пошкодження даних чи непередбачуваної поведінки. Це досягається шляхом внутрішньої синхронізації операцій. Приклади таких колекцій у `text.NET` (простір імен `textSystem.Collections.Concurrent`): `textConcurrentDictionary<TKey, TValue>`, `textConcurrentQueue<T>`, `textConcurrentStack<T>` та `textBlockingCollection<T>`.

Для чого використовуються примітиви синхронізації, такі як `textlock`, `textSemaphore`, `textAutoResetEvent`?

Відповідь

Примітиви синхронізації використовуються для координації потоків і контролю доступу до спільних ресурсів, щоб уникнути станів гонки. `textlock` забезпечує взаємне виключення (лише один потік у критичній секції). `textSemaphore` обмежує доступ до ресурсу максимальній кількості `N` потоків одночасно. `textAutoResetEvent` використовується для сигналізації, дозволяючи одному потоку повідомити іншим про завершення події, після чого він автоматично скидається.

Як забезпечити безпеку при одночасному зверненні кількох потоків до спільного ресурсу?

Відповідь

Для забезпечення безпеки спільного ресурсу необхідно використовувати механізми синхронізації. Це включає:

- 1) Блокування доступу за допомогою `textlock` навколо коду, що модифікує ресурс;
- 2) Використання `textThread-safe` колекцій, якщо ресурс є колекцією;
- 3) Застосування атомарних операцій (через `textInterlocked`) для простих типів;

4) Використання імутабельності (незмінності) об'єктів.

Як за допомогою textLINQ можна отримати мінімальне, максимальне та середнє значення певної властивості?

Відповідь

Для отримання цих значень використовуються агрегуючі методи textLINQ. Для обчислення мінімального, максимального та середнього значення певної властивості необхідно викликати методи textMin(), textMax() та textAverage() відповідно, передаючи лямбда-вираз, який вказує на потрібну властивість: наприклад, textcollection.Min(item => item.Property) для мінімального значення властивості textProperty.

У чому різниця між методами textSelect, textWhere, textAggregate, textOrderBy?

Відповідь

textSelect виконує проєкцію, перетворюючи кожен елемент послідовності в іншу форму (наприклад, вибирає лише одну властивість).
textWhere виконує фільтрацію, вибираючи лише ті елементи, які задовольняють певній умові.
textAggregate виконує агрегацію або згортання, обчислюючи одне значення з цілої послідовності (наприклад, суму, або конкатенацію).
textOrderBy виконує сортування послідовності на основі ключа, який ви передаєте.

Які переваги використання textLINQ у порівнянні з класичними циклами?

Відповідь

Основні переваги textLINQ — це читабельність та декларативність коду (ви описуєте, що хочете отримати, а не як). Він також забезпечує типову безпеку, оскільки запити перевіряються на етапі компіляції, та уніфікований синтаксис для запиту даних із різних джерел (об'єктів, баз даних, XML). Це зменшує ймовірність помилок та спрощує підтримку коду.

Що станеться, якщо два потоки одночасно спробують зберегти колекцію у файл?

Відповідь

Якщо два потоки одночасно спробують записати дані в один і той самий файл без будь-якої синхронізації, це призведе до стану гонки. Результатом буде, ймовірно, або пошкодження файлу (дані обох потоків будуть перемішані непередбачувано), або виникне `textIOException` (операційна система заблокує доступ до файлу, оскільки він уже використовується іншим процесом/потоком).

Як працює `textParallel.For` і у яких випадках його краще використовувати?

Відповідь

`textParallel.For` — це метод із `textTask Parallel Library` (TPL), який автоматично розбиває ітерації стандартного циклу `textfor` на частини і виконує ці частини паралельно, використовуючи потоки з `textThreadPool`. Його краще використовувати у випадках, коли ви маєте `textCPU-bound` (інтенсивні обчислювальні) завдання, і коли ітерації циклу є незалежними одна від одної.

Що таке пагінація і як вона реалізована у вашій системі?

Відповідь

Пагінація — це процес розбиття великого набору даних на невеликі, послідовні сторінки для ефективного відображення та обробки, що покращує продуктивність та користувацький досвід. У типовій системі пагінація реалізується на рівні бази даних або сховища даних за допомогою методів `textSkip` та `textTake` `textLINQ` (або еквівалентних команд `textSQL` `textOFFSET` та `textLIMIT`), щоб отримувати лише необхідний діапазон записів, а не весь набір даних.